CENTRO UNIVERSITÁRIO FILADÉLFIA - UNIFIL

CURSO DE ENGENHARIA DE SOFTWARE

DESENVOLVIMENTO DE PROJETOS II

GABRIEL DINIZ

GABRIEL SATIN

PETAGENDA: SISTEMA WEB DE AGENDAMENTO PARA PET SHOP

Parte II - Implementação do sistema

LONDRINA

2026

SUMÁRIO

1 INTRODUÇÃO 3

1.1 Tecnologias utilizadas 3

2 DESENVOLVIMENTO 4

2.1 Requisitos funcionais 4

2.2 Requisitos não funcionais 4

2.3 Diagrama de casos de uso 5

2.4 Diagrama Entidade-Relacionamento (DER) 6

2.5 Arquitetura e organização do código 7

2.6 Mapeamento dos casos de uso para as rotas 7

2.7 Acesso ao banco de dados 8

2.8 Regras de negócio implementadas 8

2.9 Telas do sistema 9

2.10 Testes 13

2.11 Como executar 14

2.12 Repositório 14

3 CONCLUSÃO 14

REFERÊNCIAS 15

1 INTRODUÇÃO

Pet shops de bairro costumam marcar banho, tosa e consultas pelo WhatsApp ou numa agenda de papel. Isso funciona enquanto o movimento é pequeno, mas logo aparecem problemas: dois pets marcados no mesmo horário, cliente que não aparece porque ninguém confirmou, e nenhuma ideia de quanto foi faturado no mês.

O PetAgenda é um sistema web simples para resolver isso. Nele o atendente cadastra os clientes (tutores), os pets de cada cliente e os serviços oferecidos, e faz os agendamentos. O sistema não deixa marcar dois atendimentos no mesmo horário, nem no passado, nem fora do horário de funcionamento. Na tela inicial aparece a agenda do dia, com a opção de marcar o atendimento como concluído.

Este documento corresponde à Parte II do projeto, que trata da implementação do sistema de acordo com a documentação elaborada na Parte I. São apresentados novamente os requisitos e os principais diagramas, seguidos da descrição de como o sistema foi construído, das telas e dos testes.

O trabalho foi desenvolvido em dupla pelos alunos Gabriel Diniz e Gabriel Satin. O código-fonte está no repositório público https://github.com/Gabriel-diniz007/petagenda.

1.1 Tecnologias utilizadas

Quadro 1 - Stack do projeto

Tecnologia	**Uso no projeto**
Node.js 24	Ambiente de execução do JavaScript no servidor
Express 4	Framework web: rotas, formulários e arquivos estáticos
EJS	Templates HTML renderizados no servidor
SQLite (node:sqlite)	Banco de dados relacional em arquivo, usando o módulo que já vem no Node
HTML5 e CSS3	Interface do usuário
node:test	Testes automatizados
Git e GitHub	Versionamento e trabalho em dupla

Fonte: os autores (2026).

Escolhemos o Node.js porque é uma tecnologia muito usada no mercado e permite usar JavaScript tanto no front quanto no back-end. O SQLite foi escolhido por não precisar de servidor de banco instalado: o banco é um arquivo criado automaticamente na primeira execução. A partir do Node 22 existe o módulo node:sqlite, então nem foi preciso instalar driver de banco.

2 DESENVOLVIMENTO

A seguir são retomados os requisitos e os diagramas definidos na Parte I, que guiaram a implementação, e depois é descrito como o sistema foi construído.

2.1 Requisitos funcionais

Quadro 2 - Requisitos funcionais

Código	Descrição
RF01	Cadastrar, listar, buscar (por nome ou telefone), editar e excluir clientes (tutores).
RF02	Cadastrar, listar, editar e excluir pets, vinculando cada pet a um cliente.
RF03	Cadastrar, listar, editar e excluir os serviços oferecidos, com preço e duração.
RF04	Agendar atendimentos informando pet, serviço, data e hora.
RF05	Editar e excluir agendamentos, e filtrar a lista por data e por status.
RF06	Marcar um agendamento como concluído ou cancelado.
RF07	Exibir na tela inicial a agenda do dia, o total de clientes, de pets, de agendamentos em aberto e o faturamento do mês (atendimentos concluídos).
RF08	Impedir dois agendamentos em aberto no mesmo dia e horário.
RF09	Impedir agendamento em data/hora que já passou e fora do horário de funcionamento (08h às 18h).
RF10	Impedir a exclusão de cliente com pets, de pet com agendamento em aberto e de serviço já utilizado.
RF11	Mostrar a ficha do cliente com a lista dos seus pets e a idade de cada um.

Fonte: os autores (2026).

2.2 Requisitos não funcionais

Quadro 3 - Requisitos não funcionais

Código	Descrição
RNF01	Aplicação web, acessada pelo navegador.
RNF02	Back-end em JavaScript com Node.js (versão 22.13 ou superior) e framework

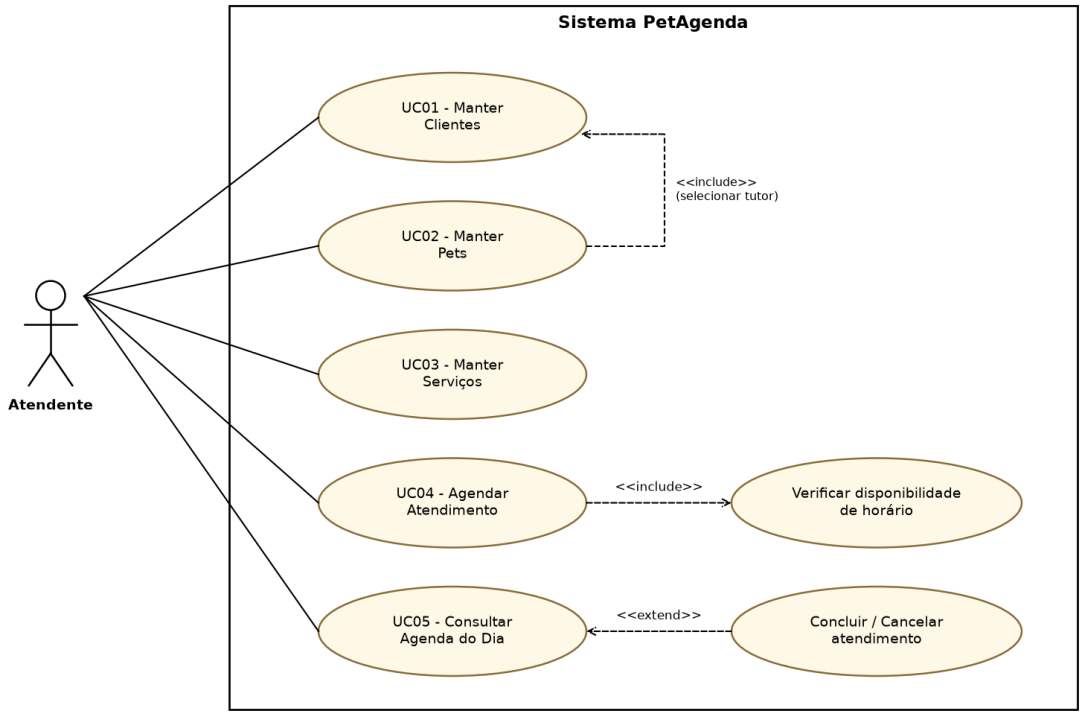
	Express.
RNF03	Dados armazenados em banco relacional SQLite, usando o módulo nativo node:sqlite.
RNF04	Interface em português, com layout responsivo para telas pequenas.
RNF05	Consultas SQL parametrizadas (prepared statements) para evitar SQL Injection.
RNF06	Mensagens claras de sucesso e erro após cada operação.
RNF07	Código versionado no GitHub, com testes automatizados das principais regras.

Fonte: os autores (2026).

2.3 Diagrama de casos de uso

O sistema tem um ator, o Atendente, que é o funcionário que recebe os clientes e organiza a agenda. Foram definidos cinco casos de uso principais, mostrados na Figura 1.

Figura 1 - Diagrama de casos de uso



Fonte: os autores (2026).

Quadro 4 - Descrição dos casos de uso

Caso de uso	Ator	Descrição
UC01 - Manter Clientes	Atendente	Cadastra, consulta, altera e exclui os tutores dos

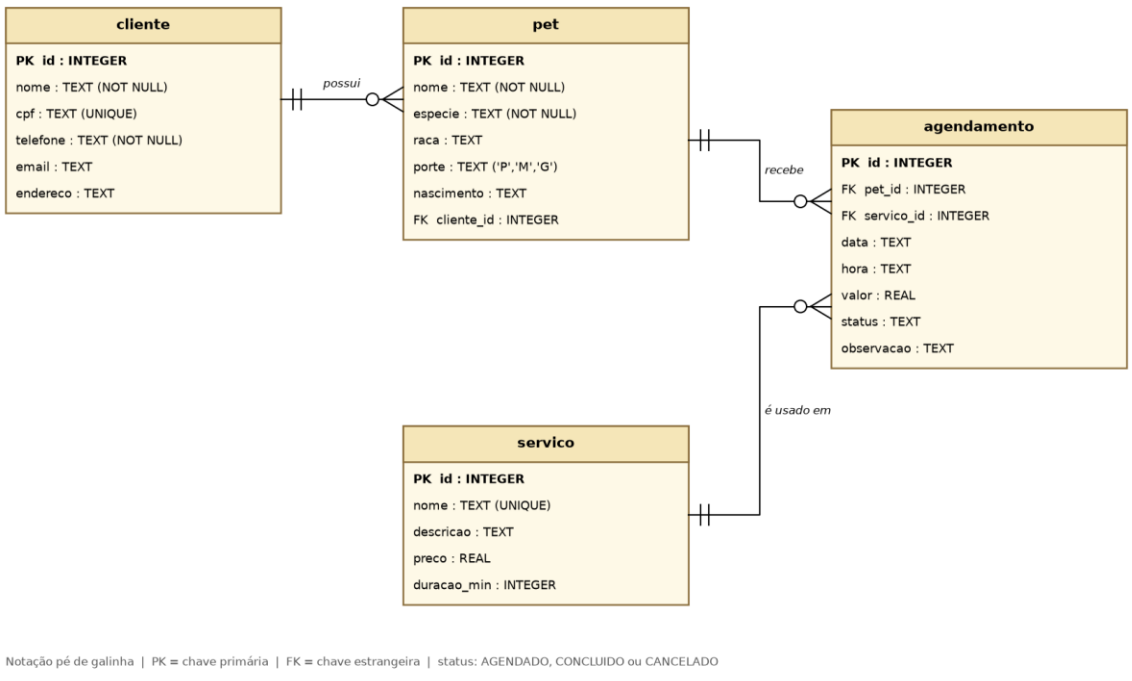
		pets.
UC02 - Manter Pets	Atendente	Cadastra, consulta, altera e exclui pets. Inclui a seleção do tutor.
UC03 - Manter Serviços	Atendente	Mantém a tabela de serviços (banho, tosa, consulta etc.).
UC04 - Agendar Atendimento	Atendente	Marca um serviço para um pet. Inclui a verificação de disponibilidade do horário.
UC05 - Consultar Agenda do Dia	Atendente	Mostra os atendimentos do dia e os indicadores. Pode ser estendido por Concluir/Cancelar atendimento.

Fonte: os autores (2026).

2.4 Diagrama Entidade-Relacionamento (DER)

O banco possui quatro tabelas. Um cliente pode ter vários pets, e todo pet pertence a um cliente. Cada agendamento liga um pet a um serviço. O valor é copiado do serviço no momento do agendamento, para que uma mudança de preço depois não altere atendimentos antigos. A coluna status aceita apenas AGENDADO, CONCLUIDO ou CANCELADO (restrição CHECK).

Figura 2 - Diagrama Entidade-Relacionamento



Fonte: os autores (2026).

2.5 Arquitetura e organização do código

O projeto segue uma separação parecida com o padrão MVC. As classes em src/models fazem o acesso ao banco e contêm as regras de negócio. Os arquivos em src/routes recebem as requisições, validam os dados do formulário e chamam os models. As telas ficam na pasta views, em templates EJS.

Quadro 5 - Estrutura de pastas

Arquivo / pasta	**Responsabilidade**
src/app.js	Configuração do Express, painel inicial e registro das rotas
src/database.js	Abre o banco SQLite, cria as tabelas e cadastra serviços iniciais
src/models/	Classes Cliente, Pet, Servico e Agendamento
src/routes/	Rotas de CRUD de cada entidade
views/	Telas em EJS (listas, formulários e painel)
public/css/style.css	Estilo visual
database/schema.sql	Script de criação das tabelas
test/app.test.js	Testes automatizados
docs/	Documentação, diagramas e prints das telas

Fonte: os autores (2026).

2.6 Mapeamento dos casos de uso para as rotas

Cada caso de uso da Parte I virou um conjunto de rotas no Express, conforme o quadro abaixo.

Quadro 6 - Rotas implementadas

Caso de uso	**Rotas**
UC01 - Manter Clientes	GET /clientes, GET/POST /clientes/novo, GET /clientes/:id, GET/POST /clientes/:id/editar, POST /clientes/:id/excluir
UC02 - Manter Pets	GET /pets, GET/POST /pets/novo, GET/POST /pets/:id/editar, POST /pets/:id/excluir
UC03 - Manter Serviços	GET /servicos, GET/POST /servicos/novo, GET/POST /servicos/:id/editar, POST /servicos/:id/excluir
UC04 - Agendar Atendimento	GET /agendamentos, GET/POST /agendamentos/novo, GET/POST /agendamentos/:id/editar, POST /agendamentos/:id/excluir
UC05 - Consultar Agenda	GET / (painel) e POST /agendamentos/:id/status (concluir/cancelar)

Fonte: os autores (2026).

2.7 Acesso ao banco de dados

A conexão é aberta uma única vez no arquivo database.js usando a classe DatabaseSync do módulo node:sqlite. Logo depois o script schema.sql é executado com CREATE TABLE IF NOT EXISTS, então o banco é criado sozinho na primeira vez que o sistema roda. Todas as consultas usam parâmetros (?), como no exemplo da verificação de conflito de horário:

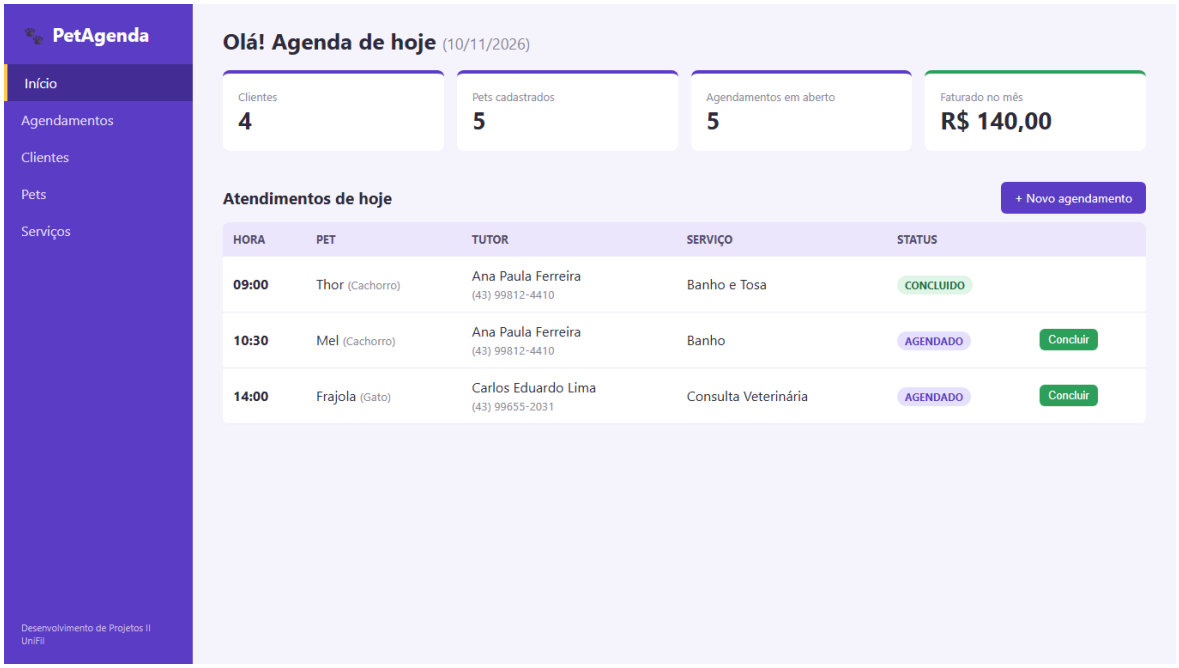
```
const conflito = db.prepare(`
  SELECT COUNT(*) AS total FROM agendamento
  WHERE data = ? AND hora = ? AND status = 'AGENDADO' AND id <> ?
`).get(this.data, this.hora, this.id || 0).total;
if (conflito > 0) erros.push('Já existe um agendamento nesse dia e horário.');
```

2.8 Regras de negócio implementadas

- Não é possível agendar em data/hora que já passou (RF09).
- Os agendamentos devem ficar entre 08:00 e 18:00 (RF09).
- Dois agendamentos com status AGENDADO não podem ocupar o mesmo dia e horário (RF08).
- O valor do agendamento é copiado do preço do serviço no momento em que é marcado.
- Cliente com pets, pet com agendamento em aberto e serviço já utilizado não podem ser excluídos (RF10).
- CPF e nome do serviço são únicos no banco (restrição UNIQUE), e o sistema avisa quando há duplicidade.
- O faturamento do mês soma apenas os atendimentos com status CONCLUIDO (RF07).

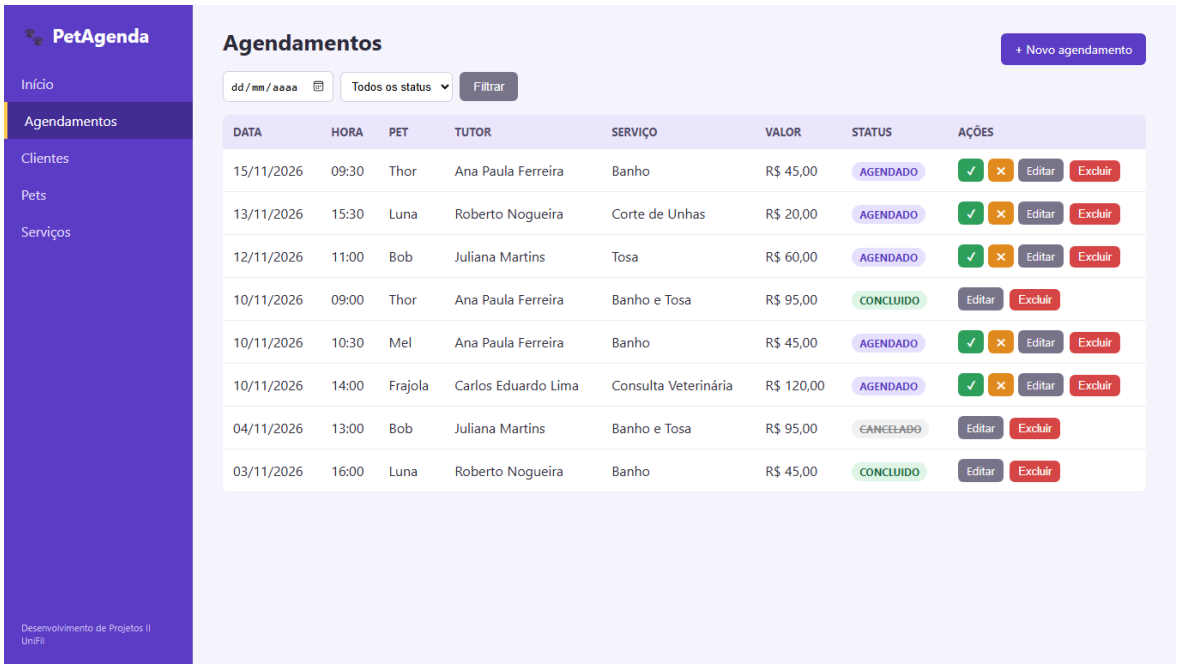
2.9 Telas do sistema

Figura 3 - Painel inicial com a agenda do dia e os indicadores (UC05)



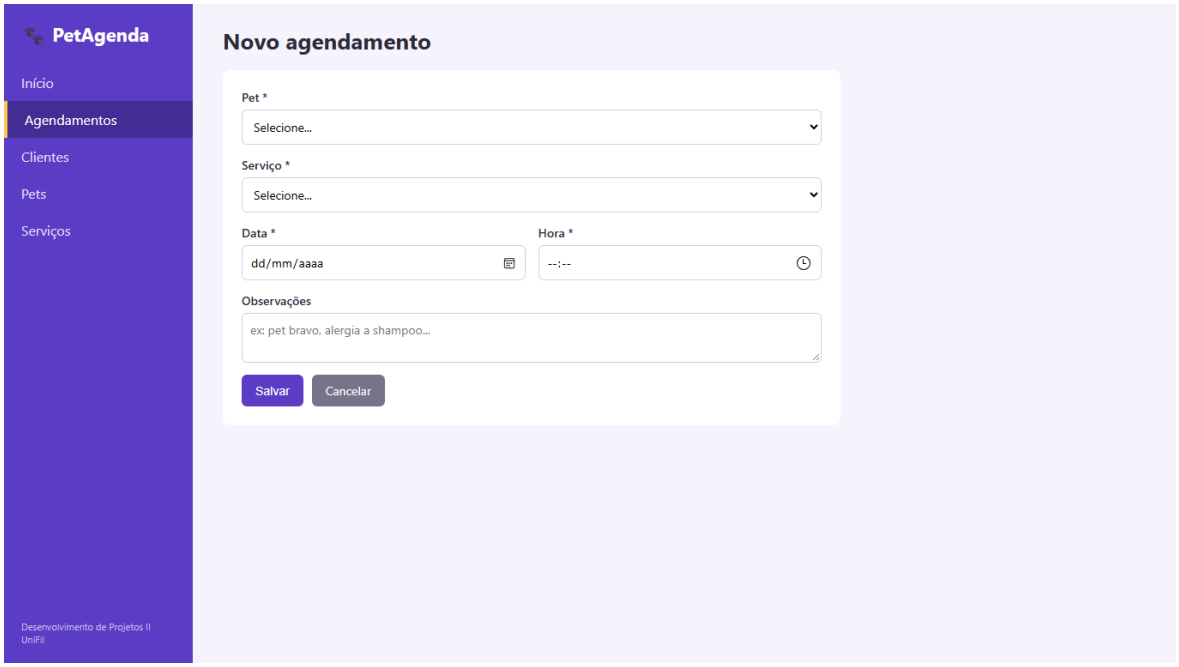
Fonte: os autores (2026).

Figura 4 - Lista de agendamentos com filtros e ações de concluir/cancelar (UC04)



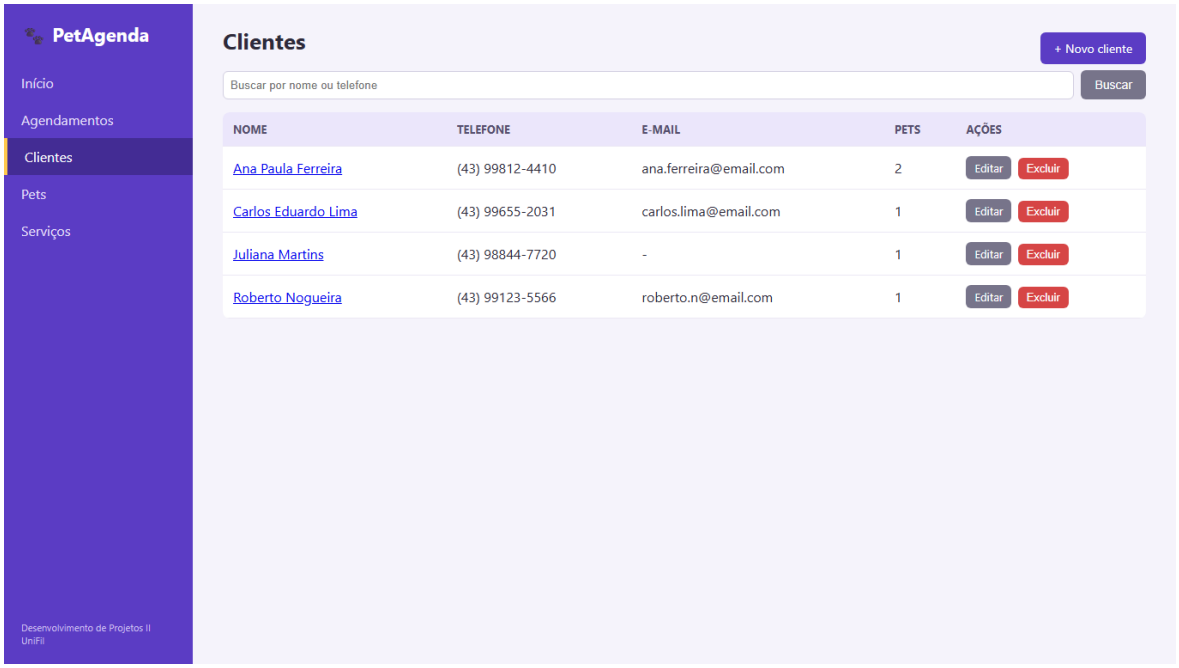
Fonte: os autores (2026).

Figura 5 - Formulário de novo agendamento (UC04)



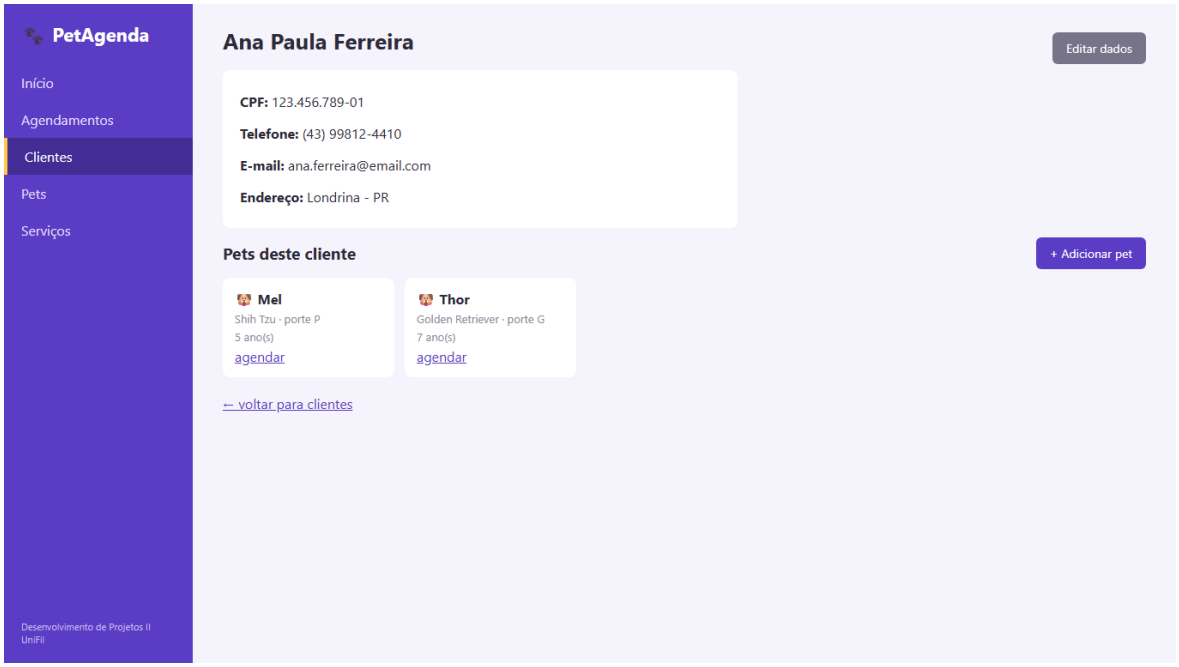
Fonte: os autores (2026).

Figura 6 - Lista de clientes com busca (UC01)



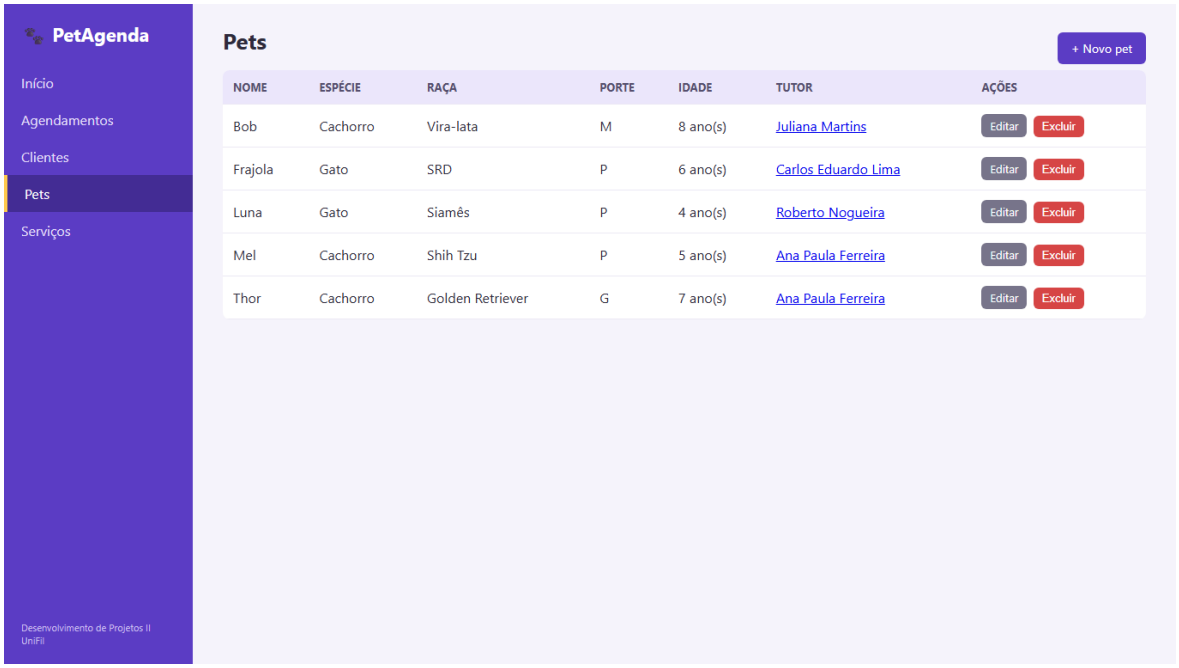
Fonte: os autores (2026).

Figura 7 - Ficha do cliente com os seus pets (UC01 e UC02)



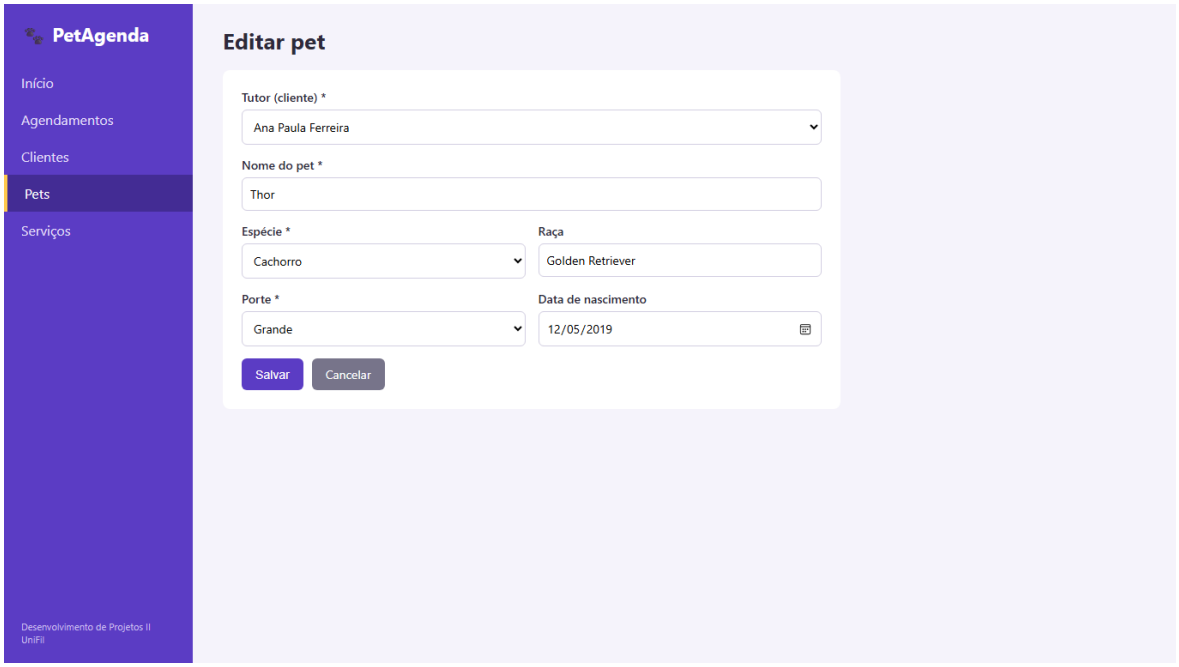
Fonte: os autores (2026).

Figura 8 - Lista de pets (UC02)



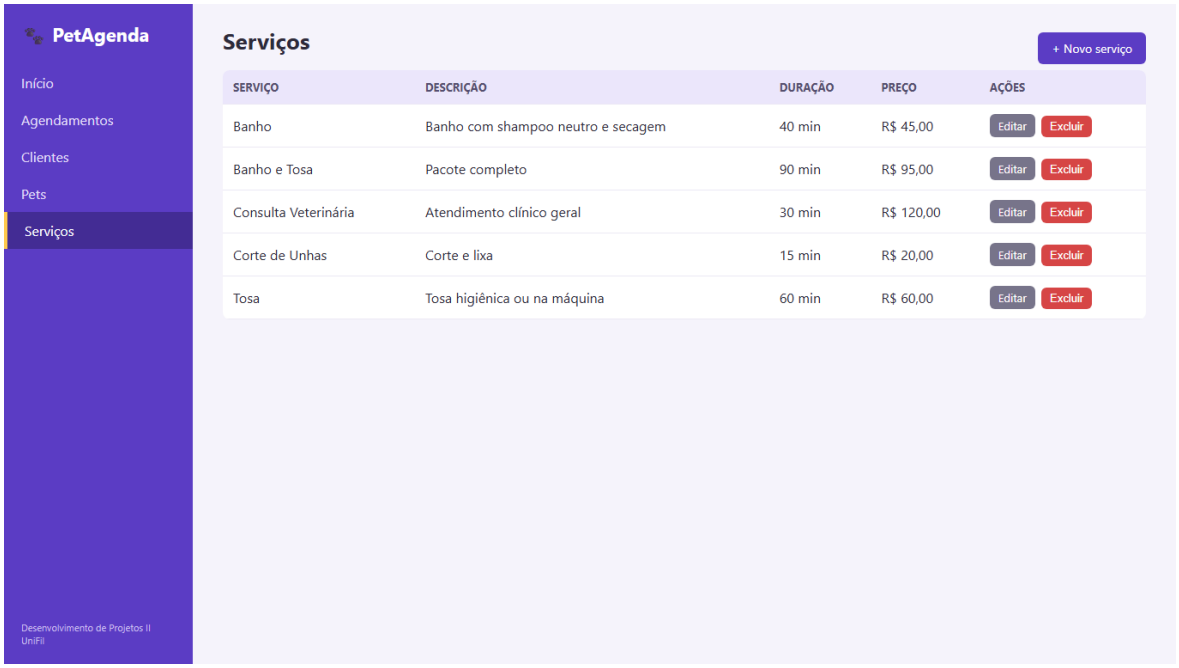
Fonte: os autores (2026).

Figura 9 - Formulário de edição de pet (UC02)



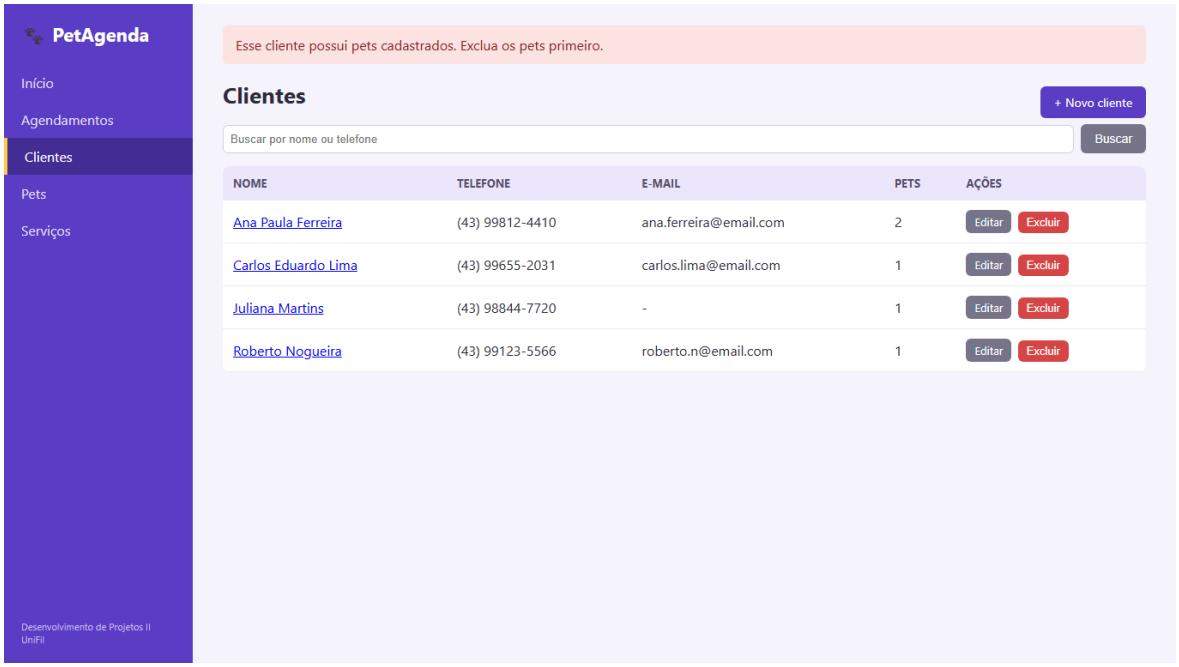
Fonte: os autores (2026).

Figura 10 - Lista de serviços (UC03)



Fonte: os autores (2026).

Figura 11 - Mensagem ao tentar excluir cliente que possui pets (RF10)



Fonte: os autores (2026).

2.10 Testes

Para conferir se as regras funcionavam sem precisar testar tudo na mão a cada alteração, criamos testes automatizados com o node:test, que já vem no Node. Os testes sobem o servidor com um banco temporário e fazem requisições reais, cobrindo o CRUD das quatro entidades e as regras de negócio. Para rodar basta usar o comando npm test. Todos os 5 testes passaram.

Quadro 7 - Testes automatizados

Teste	**O que verifica**
todas as paginas abrem	Todas as telas respondem e uma rota inexistente retorna 404
CRUD de cliente	Validação de campos, cadastro, CPF duplicado, edição e busca
CRUD de pet	Cadastro, edição e bloqueio da exclusão do cliente que tem pet
CRUD de servico	Cadastro com preço em vírgula e nome duplicado
agendamento com regras	Conflito de horário, data passada, fora do expediente, concluir, bloqueios de exclusão e exclusões em cascata

Fonte: os autores (2026).

2.11 Como executar

É necessário ter o Node.js 22.13 ou superior instalado. Na pasta do projeto:

```
npm install
npm start

# para rodar os testes:
npm test
```

O sistema fica disponível em http://localhost:3000.

2.12 Repositório

O código está no GitHub, no repositório https://github.com/Gabriel-diniz007/petagenda. A versão entregue está marcada com a tag v1.0: https://github.com/Gabriel-diniz007/petagenda/tree/v1.0. Os commits foram feitos pelas duas contas da dupla (Gabriel-diniz007 e Gabriel-satin).

3 CONCLUSÃO

A implementação mostrou na prática a importância da documentação feita na Parte I. Como os casos de uso, o DER e os diagramas de sequência já estavam prontos, criar as tabelas, as classes e as rotas foi praticamente seguir o que tinha sido desenhado. Algumas coisas foram ajustadas durante o desenvolvimento, como a regra de horário de funcionamento, que acabou entrando nos requisitos.

Na parte técnica aprendemos a montar um servidor com Express, a organizar o código em models, rotas e views, e a usar o SQLite direto pelo Node com consultas parametrizadas. Os testes automatizados foram uma novidade que ajudou bastante: várias vezes uma alteração quebrou outra parte do sistema e os testes avisaram na hora.

Trabalhar em dupla pelo GitHub também foi um aprendizado, principalmente para combinar quem mexia em qual arquivo e evitar conflitos. Como melhorias futuras, pensamos em login com perfis de usuário, envio de lembrete por WhatsApp para o cliente, histórico de atendimentos por pet e relatórios de faturamento por período.

REFERÊNCIAS

EJS. Embedded JavaScript templating. Disponível em: https://ejs.co/. Acesso em: nov. 2026.

EXPRESS. Express 4.x - API Reference. OpenJS Foundation. Disponível em: https://expressjs.com/. Acesso em: nov. 2026.

GUEDES, Gilleanes T. A. UML 2: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

HEUSER, Carlos Alberto. Projeto de banco de dados. 6. ed. Porto Alegre: Bookman, 2009.

NODE.JS. SQLite | Node.js v24 Documentation. Disponível em: https://nodejs.org/api/sqlite.html. Acesso em: nov. 2026.

NODE.JS. Test runner | Node.js v24 Documentation. Disponível em: https://nodejs.org/api/test.html. Acesso em: nov. 2026.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

SQLITE. SQLite Documentation. Disponível em: https://www.sqlite.org/docs.html. Acesso em: nov. 2026.